Computer Architecture and System Software

Lecture 03: Integer Arithmetic and Floating Points

Instructor:
David Bosc
Applied Computer Science
University of Winnipeg

Announcements

- Assignment 1 will be due next week
  - Due Jan 30th at 11:59pm
  - After today's lecture, should be able to complete all questions

So far

- So far, using unsigned/signed ints, we have:
  - Added
  - Subtracted
  - Multiplied
  - Simple divisions via shift

- Now we must consider dividing – but this opens up the possibility of fractional results
  - Must understand how fractions work in binary

Integer Binary Division without Fractions

- 20 / 4 = 5
- Try it in binary

Integer Binary Division with Fractions

Unsigned integers

- Perform long division in binary, then discard fraction (round to zero)
  - 33/8 = 4.125 □ 4
  - 42/3 = 14
  - 55/4 = 13.75 □ 13

Signed

- Treat as unsigned
  - Convert to signed when finished

Fractional Decimal Numbers

- Decimal notation uses a representation of the form: $d_m d_{m-1} \cdots d_1 d_0 . d_{-1} d_{-2} \cdots d_{-n}$
- Each digit d_i ranges between 0 and 9
- A number d can be defined as

$$d = \sum_{i=-n}^{m} 10^i \times d_i$$

Examples:

- $(12.34)_{10}$
- $(143.789)_{10}$

Fractional Binary Numbers

- Consider a notation of the form $b_m b_{m-1} \cdots b_1 b_0 b_{-1} b_{-2} \cdots b_{-n}$
- Each binary digit (or bit) b_i ranges between 0 and 1
- A number b can be defined as

$$b = \sum_{i=-n}^{m} 2^i \times b_i$$

Examples:

- $(101.11)_2$
- $(1101.0011)_2$

Observations

- In decimal the period is called a *decimal point*
- In binary the period is now a "binary point"
- Digits left of the point are weighted by positive powers, and digits right of the point are weighted by negative powers
- Shifting point right multiplies by base
- Shifting point left divides by base
- Examples:

Another Observation

- Numbers of the form 0.11 … 1 represent values just less than one.
- e.g. $0.11111_2 = 1/2 + 1/4 + 1/8 + 1/16 + 1/32$

$$= 16/32 + 8/32 + 4/32 + 2/32 + 1/32$$

$$= 31/32$$

Recall

- Dec to Hex Conversion
  - Keep dividing by 16, the remainders become the hex digits

$$
\begin{aligned}
314156 &= 19634 \cdot 16 + 12 && \text{(C)} \\
19634 &= 1227 \cdot 16 + 2 && (2) \\
1227 &= 76 \cdot 16 + 11 && \text{(B)} \\
76 &= 4 \cdot 16 + 12 && \text{(C)} \\
4 &= 0 \cdot 16 + 4 && (4)
\end{aligned}
$$

- Dec to Binary Conversion follows same process
  - Just replace 16 with 2

Conversion of binary fractions

- Process left and right separately
- Process might not terminate (look for repeating pattern)
- For right of point, we will use the inverse of the previous process

Conversion

- Keep multiplying fraction by 2, 'storing' ones whenever the result >= 1. Store zero otherwise. Keep working on fractional parts until you are done.
- Examples

  0.375:

  0.375 * 2 = 0.75 (0)

  0.75 * 2 = 1.5 (1)

  0.5 * 2 = 1 (1)

  Read results from top to bottom this time! (Inverse process)

  - Try 10.3125, and 0.3.

Representable Numbers

- Each base has numbers that are not representable with finite number of digits
  - Decimal
    - Finite: 1/10 = 0.1
    - Infinite: 1/3 = 0.33 [3] …
  - Base 3
    - Finite: 1/3 = 0.1
    - Infinite: 1/10 = 0.00220022 [0022] …
  - Binary
    - Finite: 1/2 = 0.1
    - Infinite: 1/3 = 0.0101 [01] …
- Limitation is acceptable (usually, an approximation is sufficient)

Multiplying fractions

- Base 10:
  - Count total # of digits to right of decimal point in multiplicands
  - Total # of digits to right of decimal point is conserved in the result

3.3 x 0.05 = 0.165

OR

33 x 5 = 165 □ shift 3 decimal places □ 0.165

- Base 2: Same procedure

Dividing fractions

- Base 10:
  - Multiply dividend AND divisor by power of 10 until there is no decimal

    1.004 / 0.02 = 1004 / 20

  - Now use long division as normal

- Base 2:
  - Same procedure (except we multiply by power of 2)
  - Equivalent is shifting left
  - 0.01 / 0.001 = (0.01 << 3)/(0.001 << 3) = 10/1

Chapter 2

Section 2.4 Floating Point

Floating Point

- Finite-length encodings cannot represent numbers such as 1/3 and 5/7
- Fractional binary notation can only represent numbers in the form $V = M \times 2^y$
- Examples:
  - $5 \times 2^{-3} = 5/8 = 0.101$
  - $5 \times 2^{-4} = 5/16 = 0.0101$
  - $5 \times 2^{-8} = 5/256 = 0.00000101$
- Notice: y determines location of point
  - Only need to store M and y
- Hence: floating point notation

Floating Point Representation

- Encode values as $(-1)^s \times M \times 2^E$
  - Sign bit **s** determines whether number is negative or positive
  - Significand **M** fractional binary number ranging from [1 2), or from [0 and 1)
  - Exponent **E** weights value by +ve or –ve power of 2

Floating Point Encoding

- Bit representation
  - Single bit encodes sign
  - `exp` contains k bits encoding **E**
  - `frac` contains n bits encoding **M**
  - Total number of bits = 1+k+n
- Standards
  - Single precision: 8 `exp` bits, 23 `frac` bits
    - 32 bits total
  - Double precision: 11 `exp` bits, 52 `frac` bits
    - 64 bits total

Floating Point Encodings

- Floating point standard specifies three different categories for a value encoded by a given bit representation
  - Normalized: majority of all values
  - Denormalized: values near and including 0
  - Special: $+\infty$, $-\infty$, NaN

Normalized Values

- Most common case
- Occurs when `exp` is neither all 0s or all 1s
  - *i.e.* $0 < \texttt{exp} < 2^k - 1$
- **E** is interpreted as representing a signed integer in biased form
  - **E** = `exp` – bias
- `exp` is an unsigned number
- Bias = $2^{k-1} - 1$ (127 or 1023 for single and double)

Normalized Values Continued

- Bits in `frac` are interpreted as $0.f_{n-1} \ldots f_1 f_0$
- And **M** is interpreted as 1 + `frac`
  - This is called an implied leading 1 because **M** can be viewed as $1.f_{n-1} \ldots f_1 f_0$
  - Trick for getting an extra bit of precision for free

Denormalized Values

- Occurs when `exp` is all 0s
  - *i.e.* `exp` = 00 … 0
- **E** is interpreted as **E** = 1 – bias
- **M** is interpreted as `frac` without implied leading 1
  - *i.e.* **M** = $0.f_{n-1} \ldots f_1 f_0$

Denormalized Values Continued

Why use denormalized values ?

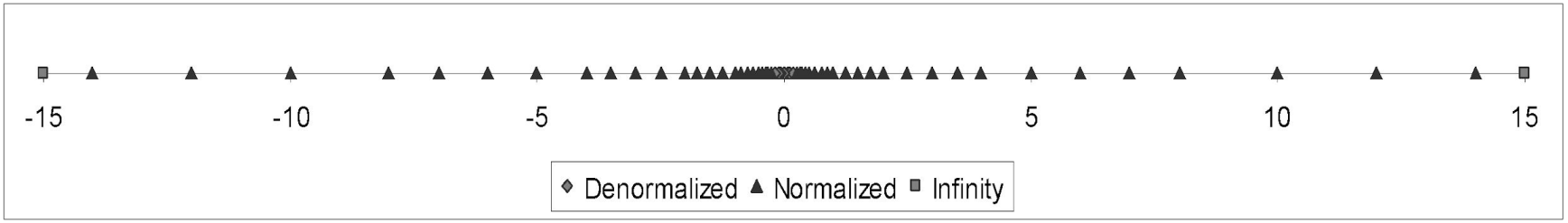
- Represent 0 (not possible with normalized values)
  - **M** >= 1 and $(-1)^s \times M \times 2^E$
- Represent numbers very close to 0
  - Provides a property called gradual underflow in which possible numeric values are spaced evenly near 0

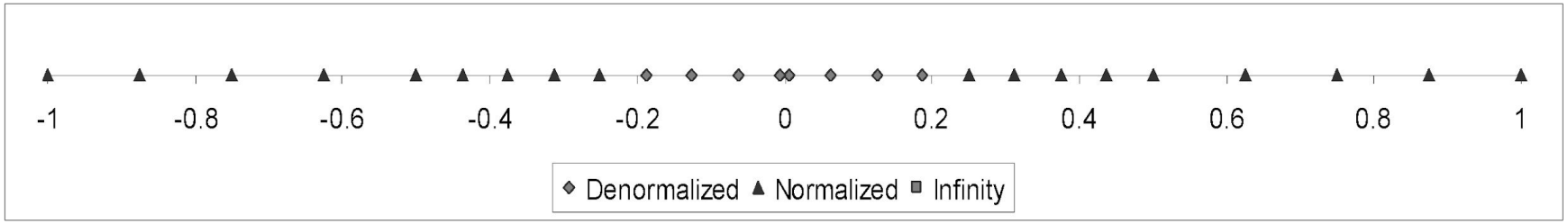
Special Values

- Occurs when `exp` is all 1s
  - *i.e.* `exp` = 11 … 1
- When `frac` is all 0s
  - +∞ when s = 0
  - -∞ when s = 1
- Infinity can represent overflow
  - Multiply two very large numbers
  - Divide by 0
- When `frac` is non-zero we get NaN
  - When sqrt(-1) or ∞ – ∞

First Example

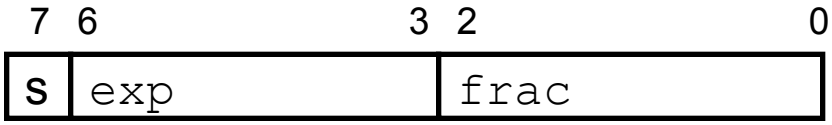
- k = 3 and n = 2 (bias is 3)
- Top shows representable values (max is +- 14)
- Denormalized values clustered around 0
- Representable numbers are not uniformly distributed





Second Example

- Let's consider a small floating point type where k = 4 and n = 3



Second Example Continued

	s	exp	frac	*E*	Value	
Denormalized numbers	0	0000	000	-6	0	
	0	0000	001	-6	1/8*1/64 = 1/512	← closest to zero
	0	0000	010	-6	2/8*1/64 = 2/512	
	…					
	0	0000	110	-6	6/8*1/64 = 6/512	
	0	0000	111	-6	7/8*1/64 = 7/512	← largest denorm
Normalized numbers	0	0001	000	-6	8/8*1/64 = 8/512	← smallest norm
	0	0001	001	-6	9/8*1/64 = 9/512	
	…					
	0	0110	110	-1	14/8*1/2 = 14/16	
	0	0110	111	-1	15/8*1/2 = 15/16	← closest to 1 below
	0	0111	000	0	8/8*1 = 1	
	0	0111	001	0	9/8*1 = 9/8	← closest to 1 above
	0	0111	010	0	10/8*1 = 10/8	
	…					
	0	1110	110	7	14/8*128 = 224	
	0	1110	111	7	15/8*128 = 240	← largest norm
	0	1111	000	n/a	inf	

Comments on Previous Example

- Denormalized numbers are closest to 0
  - Positive numbers in the interval [0, 7/512]
- Observe smooth transition
  - Due to choice of **E** = 1- bias for denormalized values
- Positive normalized numbers
  - Below one in the interval [8/512, 480/512]
- Interesting property is that bit patterns of previous slide occur in ascending order
  - Intentional: Designed so that FP could be sorted using integer-sorting routines

Observations

- Maximum FP value (6 exp bits)
  - 0 11110 111111…
- Minimum FP value (6 exp bits)
  - 1 11110 111111…

- Two representations for zero
  - 0 000000 000000…
  - 1 000000 000000…

Range of FPs

- Approximate range can be easily found by noting
  - Maximum M = $1.1111 \ldots \approx 2$
  - Maximum normalized exp = 111….10 = $2^k - 2$ (k-bits)

    $= 2(2^{k-1} - 1) = 2*\text{bias}$
  - Maximum E = $\exp_{max} - \text{bias} = \text{bias}$
  - So Range(FP) $\approx [-2 \times 2^{bias}, +2 \times 2^{bias}]$

    $\approx [-2^{bias+1}, +2^{bias+1}]$

Converting to FP Notation

- Example: 101.125 with k=8, n=9

- What happens if n=11? n=7? How much precision do we lose in this case?

Decimal Addition

- Review decimal addition in scientific notation:
  - Step 1: Match exponents by shifting decimal point
  - Step 2: Add
  - Step 3: Normalize result (if needed)
- Examples:
  - Without normalization: $(3.25 \times 10^3)+(2.63 \times 10^{-1})$
  - With normalization: $(9.25 \times 10^3)+(8.41 \times 10^2)$

Floating Point Addition

- Before proceeding note:
  - Shifting `frac` left by 1 bit decreases E/`exp` by 1
  - Shifting `frac` right by 1 bit increases E/`exp` by 1
- The idea behind adding FP numbers is to make `exp` of the smaller number equal to `exp` of the larger
- Then, the steps in binary are the same as decimal:
  - Convert numbers to binary FP (if necessary)
  - Align "binary points" by matching exp of both numbers
  - Add
  - Normalize result

Example: FP Addition

- 100 + 0.3125
- 100 + 80

Floating Point Subtraction

- Like addition in terms of aligning radix points
- But, requires a few more rules:
- Pay attention to the signs and magnitudes of operands. We always want the value of M to be a positive number after subtraction since it is an unsigned binary fraction.
- Sign of the result after subtraction is encoded by sign bit only. You shouldn't have negative integers anywhere.
- Don't forget to normalize!

Examples: FP Subtraction

- 100 – 80
  - Result is positive, so perform subtraction as normal
- 100 – (-80)
  - Simplifies to 100 + 80. Do addition instead.
- 80 – 100
  - Result is negative! But 80 -100 = -(-80+100).
  - Therefore, do the addition and change the sign bit to 1.
- -100 – (-80)
  - Simplifies to -(100-80). Do the subtraction and change the sign bit to 1.
- -80 – 100
  - Simplifies to - (80 + 100). Do the addition and change the sign bit to 1.

Example (on board)

- Consider the following 16-bit FP numbers (1 sign bit, 6 exp bits (k=6), 9 frac bits (n=9), bias = 31.

- R1 = 1100 0110 1000 0110
- R2 = 0100 0100 1110 0000

- Perform R1+R2 and R1-R2 in FP representation.

Floating Point Multiplication

- Multiply significands and add **exponents**
  - (add **E** not `exp`)
- Decimal
  - (3 × 10^1) (5 × 10^2) = $15 \times 10^3 = 1.5 \times 10^4$
- Binary
  - Same procedure.
  - Convert to binary FP representation when finished.

Floating Point Division

- Perform unsigned division on significands
- Subtract **exponent**
  - (subtract **E** not `exp`)
- Decimal
  - $(5 \times 10^2) / (2 \times 10^1) = (2.5 \times 10^1)$
- Binary
  - Same procedure.
  - Convert to binary FP representation when finished.

Denormalized Floating points

- Recall with normalized floating points,
  - M = 1+frac
  - E = exp-bias
  - Bias = 2^{k-1} -1

- Denormalized floating points:
  - M = frac
  - E = 1-bias
  - Bias = 2^{k-1} -1
  - exp = 0

Denormalized Floating points

- Arithmetic with denormalized floating points follows the same rules as normalized floating points except
  - E cannot go lower than E = 1-bias
  - M < 1 (hence denormalized)
  - If after arithmetic, M >=1, reinterpret result as a normalized floating point
- If performing arithmetic in FP representation and your result has E <= 1-bias
  - Set E = 1 – bias (add to E)
  - Shift M appropriately

In-class Examples

- Using floating points with k = 3 exp bits and n = 4 frac bits, (bias = 3), calculate the following:
- A) Adding two denormalized numbers and getting a denormalized result
  - 0.078125 + 0.046875
- B) Adding two denormalized numbers and getting a normalized result
  - 0.234375 + 0.234375
- C) Subtracting two normalized numbers and getting a denormalized result
  - 0.5625 - 0.375

Examples

- A) $R_1 + R_2$
- $R_1 = (0.078125)_{10} = (1.01 \times 2^{-4})_2$
  $R_2 = (0.046875)_{10} = (1.1 \times 2^{-5})_2$
  (confirm for yourself as an exercise)
- If we try to interpret as a normalized FP
  - $exp_1 = E_1 + bias = -4 + (2^{k-1}-1) = -1$
  - But exp must be an unsigned int! So we must interpret as denormalized instead
- $E_1 = 1\text{-bias} = -2, \quad M = 0.0101$
  - Similarly, $R_2 = 0.0011 \times 2^{-2}$.

Examples

- $R_1 = 0.0101 \times 2^{-2}$
  - Then $M_1 = 0.0101 = frac_1$, exp = 0
  - Similarly, $M_2 = 0.0011 = frac_2$, exp = 0
  - Add M values since exponents match
  - $M_1 + M_2 = 0.1000$
  - Do not normalize

    Result: 0 000 1000

    s exp frac

  - Questions B) ,C) on board

Floating point summary

- Binary scientific notation
- $(-1)^s \times M \times 2^E$
- Range of float grows exponentially with the number of *k* bits in exp
- Precision determined by remaining *n* bits in frac
- Float ≠ real number
- Overflow handled differently than int ($\pm\infty$)

For Next Lecture

- Introduction to Assembly Language
- Working with Intel 32-bit architecture
- For work at home you will need DOSBOX
  - http://www.dosbox.com/

- Other software packages available to simulate 32-bit architecture, but solutions given in class will only be guaranteed to compile in DOSBOX environment